

УНИВЕРСИТЕТ ИТМО

Факультет прикладной информатики
Направление подготовки 09.03.03 Прикладная информатика
Дисциплина «Мобильная разработка»

КУРСОВАЯ РАБОТА

“Разработка учебного Android-приложения «Messenger»”

Выполнил(а):
Даньшин С.А.

Группа:
К3440

Руководитель:
Шатинский Г.С.

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	2
1 Постановка задачи	2
2 Краткое описание приложения	3
3 Архитектура и проектирование	4
3.1 Архитектурный подход	5
3.2 Диаграмма компонентов	5
4 Реализация	5
4.1 Навигация Single-Activity	6
4.2 Экран ленты новостей	7
4.3 ViewModel и асинхронные операции	9
4.4 Сетевой слой (Retrofit)	10
4.5 Локальное хранилище (Room) и «лайки»	10
4.6 Отслеживание состояния сети	11
4.7 Фоновая синхронизация и уведомления	12
5 Диаграмма сценария обновления ленты	15
6 Сборка и запуск	16
ЗАКЛЮЧЕНИЕ	18
Список литературы	19
ПРИЛОЖЕНИЕ А. Фрагменты исходного кода	20
A.1 MainActivity (навигация + разрешения)	20
A.2 NewsFeedFragment (UI ленты)	22
A.3 NewsViewModel (обновление и лайки)	26
A.4 Репозиторий и локальный кэш (Room)	28
A.5 Retrofit API	30
A.6 Сеть/фон/уведомления	31
A.7 Навигационный граф	35

ВВЕДЕНИЕ

В рамках курсовой работы выполнена серия лабораторных работ по дисциплине «Мобильная разработка», реализованных в виде учебного Android-приложения «Messenger» на языке Kotlin. Приложение построено по подходу Single-Activity с использованием Navigation Component и нижней панели навигации (Bottom Navigation). Реализованы сетевое получение данных (Retrofit), локальное кэширование (Room), асинхронность (Kotlin Coroutines), фоновая периодическая синхронизация (WorkManager), уведомления о синхронизации и обработка состояния сети.

1 ПОСТАНОВКА ЗАДАЧИ

Цель работы — разработать учебное мобильное приложение, демонстрирующее типовые компоненты Android-разработки:

- навигацию и многоэкранный интерфейс (Navigation Component, Bottom Navigation);
- архитектурный подход MVVM (ViewModel + наблюдаемое состояние);
- сетевой обмен данными с REST API;
- локальное хранение данных и кэширование;
- фоновые задачи и уведомления;
- обработку жизненного цикла и состояния сети.

2 КРАТКОЕ ОПИСАНИЕ ПРИЛОЖЕНИЯ

Приложение содержит три вкладки:

- «Новости» — лента сообщений (посты), поддерживает обновление свайпом, отображение оффлайн-баннера и локальные «лайки»;
- «Профиль» — редактирование имени и статуса пользователя с синхронизацией с ViewModel;
- «Настройки» — переключатель тёмной темы и переключатель уведомлений (UI).

Источник данных для ленты — публичный REST API <https://jsonplaceholder.typicode.com/posts>.

3 АРХИТЕКТУРА И ПРОЕКТИРОВАНИЕ

3.1 Архитектурный подход

Для разделения ответственности использован подход MVVM:

- **View** — Fragment/Activity и разметки (XML);
- **ViewModel** — хранение состояния и запуск асинхронных операций;
- **Repository** — единая точка доступа к данным (сеть + база).

3.2 Диаграмма компонентов

На рисунке 1 показано взаимодействие основных компонент.

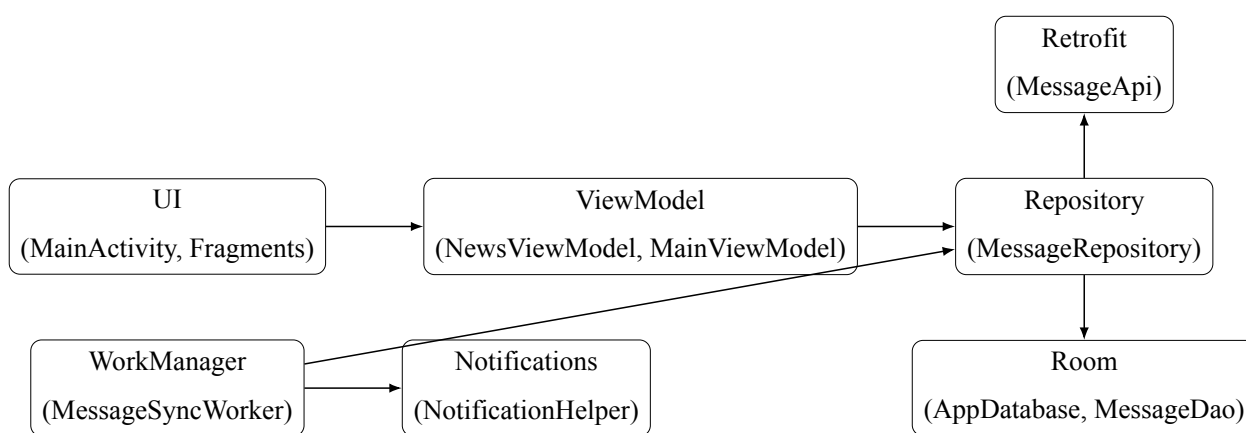


Рисунок 1 – Компоненты приложения и зависимости

4 РЕАЛИЗАЦИЯ

4.1 Навигация Single-Activity

Навигация построена на базе Navigation Component: Activity содержит NavHostFragment и BottomNavigationView.

```
1 создана
2
3     if (toRequest.isNotEmpty()) {
4         permissionsLauncher.launch(toRequest.toTypedArray())
5     }
6 }
7
8 private fun setupNavigation() {
9     val navHostFragment = supportFragmentManager
10         .findFragmentById(R.id.nav_host_fragment) as NavHostFragment
11     val navController = navHostFragment.navController
12
13     val appBarConfiguration = AppBarConfiguration(
14         setOf(
15             R.id.navigation_news,
16             R.id.navigation_profile,
17             R.id.navigation_settings
18         )
19     )
20     setupActionBarWithNavController(navController, appBarConfiguration
21 )
22     binding.bottomNavigation.setupWithNavController(navController)
23     Log.d(TAG, "setupNavigation: Навигация настроена")
24 }
25
26 override fun onSupportNavigateUp(): Boolean {
27     val navHostFragment = supportFragmentManager
28         .findFragmentById(R.id.nav_host_fragment) as NavHostFragment
29     val navController = navHostFragment.navController
30     return navController.navigateUp() || super.onSupportNavigateUp()
31 }
32
33 override fun onStart() {
```

Листинг 1 – Инициализация навигации в Activity

Полный исходный код приведён в приложении (листинг 9).

4.2 Экран ленты новостей

Экран «Новости» реализован на базе RecyclerView и SwipeRefreshLayout. При отсутствии сети отображается баннер и отключается обновление.

```
1  прикрепленкаактивностисоздан
2      savedInstanceState: Bundle?
3  ): View {
4      Log.d(TAG, "onCreateView: Создание View для Fragment")
5      _binding = FragmentNewsFeedBinding.inflate(inflater, container,
        false)
6      return binding.root
7  }
8
9  override fun onCreateView(view: View, savedInstanceState: Bundle?) {
10     super.onCreateView(view, savedInstanceState)
11     Log.d(TAG, "onViewCreated: View создан")
12
13     // Initialize Data Layer
14     val database = AppDatabase.getDatabase(requireContext())
15     val repository = MessageRepository(database.messageDao(),
        RetrofitClient.api)
16     val factory = NewsViewModelFactory(requireActivity().application,
        repository)
17     viewModel = ViewModelProvider(this, factory)[NewsViewModel::class.
        java]
18
19     // Setup RecyclerView
20     adapter = MessageAdapter { message ->
21         viewModel.toggleLike(message)
22     }
23     binding.rvMessages.layoutManager = LinearLayoutManager(context)
24     binding.rvMessages.adapter = adapter
25
26     // Observe Data
27     viewModel.messages.observe(viewLifecycleOwner) { messages ->
28         adapter.submitList(messages)
29     }
30
```



```

31     viewModel.isOnline.observe(viewLifecycleOwner) { isOnline ->
32         binding.offlineBanner.isVisible = !isOnline
33         binding.swipeRefreshLayout.isEnabled = isOnline
34         if (!isOnline) {
35             Snackbar.make(binding.root, getString(R.string.
offline_message), Snackbar.LENGTH_SHORT).show()
36         }
37     }
38
39     viewModel.isLoading.observe(viewLifecycleOwner) { isLoading ->
40         binding.swipeRefreshLayout.isRefreshing = isLoading
41     }
42
43     // Setup Listeners
44     binding.swipeRefreshLayout.setOnRefreshListener {
45         viewModel.refreshMessages()
46     }
47
48     binding.fabNewMessage.setOnClickListener {
49         Toast.makeText(requireContext(), getString(R.string.
new_message), Toast.LENGTH_SHORT).show()
50     }
51
52     // Initial load if empty (optional, but good UX)
53     if (viewModel.messages.value.isNullOrEmpty()) {
54         viewModel.refreshMessages()
55     }
56 }
57
58 override fun onStart() {
59     super.onStart()
60     Log.d(TAG, "onStart: Fragment запущен")
61 }
62
63 override fun onResume() {
64     super.onResume()
65     Log.d(TAG, "onResume: Fragment возобновлён")

```

Листинг 2 – Подписка на данные и обработка онлайн/оффлайн

4.3 ViewModel и асинхронные операции

ViewModel использует `viewModelScope` и корутины для сетевого обновления ленты и переключения лайков.

```
1
2     private val _isLoading = MutableLiveData<Boolean>()
3     val isLoading: LiveData<Boolean> = _isLoading
4
5     fun refreshMessages() {
6         viewModelScope.launch {
7             _isLoading.value = true
8             try {
9                 if (isOnline.value != false) {
10                     repository.refreshMessages()
11                 }
12             } catch (e: Exception) {
13                 // Handle error (e.g., show toast via LiveData event)
14             } finally {
15                 _isLoading.value = false
16             }
17         }
18     }
19
20     fun toggleLike(message: Message) {
21         viewModelScope.launch {
22             repository.toggleLike(message)
23         }
24     }
25 }
26
27 class NewsViewModelFactory(
28     private val application: Application,
29     private val repository: MessageRepository
30 ) : ViewModelProvider.Factory {
31     override fun <T : ViewModel> create(modelClass: Class<T>): T {
32         if (modelClass.isAssignableFrom(NewsViewModel::class.java)) {
33             @Suppress("UNCHECKED_CAST")
34             return NewsViewModel(application, repository) as T
35         }
36         throw IllegalArgumentException("Unknown ViewModel class")
37     }
38 }
```

```

37     }
38 }

```

Листинг 3 – Асинхронное обновление ленты во ViewModel

4.4 Сетевой слой (Retrofit)

Для работы с REST API используется Retrofit и конвертер Gson.

```

1 package com.example.messenger.data.api
2
3 import com.example.messenger.data.model.Message
4 import retrofit2.http.GET
5
6 interface MessageApi {
7     @GET("posts")
8     suspend fun getMessages(): List<Message>
9 }

```

Листинг 4 – Retrofit API для получения сообщений

4.5 Локальное хранилище (Room) и «лайки»

Для кэширования сообщений применяется Room. При обновлении ленты выполняется объединение удалённых данных с локальным состоянием лайков.

```

1 package com.example.messenger.data.repository
2
3 import com.example.messenger.data.api.MessageApi
4 import com.example.messenger.data.db.MessageDao
5 import com.example.messenger.data.model.Message
6 import kotlinx.coroutines.flow.Flow
7
8 class MessageRepository(
9     private val messageDao: MessageDao,
10    private val messageApi: MessageApi
11 ) {
12    val messages: Flow<List<Message>> = messageDao.getAllMessages()
13
14    suspend fun refreshMessages() {
15        try {
16            val localLikes = messageDao.getMessageLikes().associateBy { it
17                .id }
18            val remoteMessages = messageApi.getMessages()
19
20            // ...
21        }
22    }
23 }

```

```

18         val mergedMessages = remoteMessages.map { message ->
19             val isLiked = localLikes[message.id]?.isLiked ?: false
20             message.copy(isLiked = isLiked)
21         }
22         messageDao.insertAll(mergedMessages)
23     } catch (e: Exception) {
24         // Handle error or rethrow. For now, we just log or ignore to
        keep UI simple,
25         // but in a real app we'd expose an error state.
26         e.printStackTrace()
27         throw e
28     }
29 }
30
31 suspend fun toggleLike(message: Message) {
32     messageDao.updateLike(message.id, !message.isLiked)
33 }

```

Листинг 5 – Репозиторий: объединение удалённых данных и локальных лайков

4.6 Отслеживание состояния сети

Состояние сети отслеживается через `ConnectivityManager` и транслируется во `Flow<Boolean>`.

```

1 package com.example.messenger.data.network
2
3 import android.content.Context
4 import android.net.ConnectivityManager
5 import android.net.Network
6 import android.net.NetworkCapabilities
7 import android.net.NetworkRequest
8 import kotlinx.coroutines.channels.awaitClose
9 import kotlinx.coroutines.flow.Flow
10 import kotlinx.coroutines.flow.callbackFlow
11 import kotlinx.coroutines.flow.distinctUntilChanged
12
13 class NetworkStatusTracker(context: Context) {
14     private val connectivityManager =
15         context.getSystemService(Context.CONNECTIVITY_SERVICE) as
        ConnectivityManager
16

```

```

17     val isOnline: Flow<Boolean> = callbackFlow {
18         val callback = object : ConnectivityManager.NetworkCallback() {
19             override fun onAvailable(network: Network) {
20                 trySend(true)
21             }
22
23             override fun onLost(network: Network) {
24                 trySend(false)
25             }
26         }
27
28         val activeNetwork = connectivityManager.activeNetwork
29         val capabilities = connectivityManager.getNetworkCapabilities(
30             activeNetwork)
31         val connected = capabilities?.hasCapability(NetworkCapabilities.
32             NET_CAPABILITY_INTERNET) == true
33         trySend(connected)
34
35         val request = NetworkRequest.Builder()
36             .addCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET)
37             .build()
38
39         connectivityManager.registerNetworkCallback(request, callback)
40
41         awaitClose {
42             connectivityManager.unregisterNetworkCallback(callback)
43         }
44     }.distinctUntilChanged()
45 }

```

Листинг 6 – Трекер состояния сети на базе callbackFlow

4.7 Фоновая синхронизация и уведомления

Фоновая синхронизация выполняется периодически каждые 15 минут через WorkManager при наличии сети. По успешной синхронизации показывается уведомление.

```

1 package com.example.messenger.data.work
2
3 import android.content.Context
4 import androidx.work.Constraints

```

```

5 import androidx.work.ExistingPeriodicWorkPolicy
6 import androidx.work.NetworkType
7 import androidx.work.PeriodicWorkRequestBuilder
8 import androidx.work.WorkManager
9 import java.util.concurrent.TimeUnit
10
11 object WorkScheduler {
12     private const val WORK_NAME = "message_sync_work"
13
14     fun scheduleMessageSync(context: Context) {
15         val constraints = Constraints.Builder()
16             .setRequiredNetworkType(NetworkType.CONNECTED)
17             .build()
18
19         val request = PeriodicWorkRequestBuilder<MessageSyncWorker>(15,
            TimeUnit.MINUTES)
20             .setConstraints(constraints)
21             .build()
22
23         WorkManager.getInstance(context)
24             .enqueueUniquePeriodicWork(WORK_NAME,
                ExistingPeriodicWorkPolicy.UPDATE, request)
25     }
26 }

```

Листинг 7 – Планирование периодической синхронизации (WorkManager)

```

1 package com.example.messenger.data.notification
2
3 import android.Manifest
4 import android.app.NotificationChannel
5 import android.app.NotificationManager
6 import android.content.Context
7 import android.content.pm.PackageManager
8 import android.os.Build
9 import androidx.core.app.NotificationCompat
10 import androidx.core.app.NotificationManagerCompat
11 import androidx.core.content.ContextCompat
12 import com.example.messenger.R
13
14 object NotificationHelper {
15     const val CHANNEL_ID = "sync_channel"

```

```

16     private const val CHANNEL_NAME = "Синхронизация"
17     private const val CHANNEL_DESCRIPTION = "Уведомления о синхронизации
сообщений"
18
19     fun createChannel(context: Context) {
20         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
21             val channel = NotificationChannel(
22                 CHANNEL_ID,
23                 CHANNEL_NAME,
24                 NotificationManager.IMPORTANCE_DEFAULT
25             ).apply {
26                 description = CHANNEL_DESCRIPTION
27             }
28             val manager = context.getSystemService(Context.
NOTIFICATION_SERVICE) as NotificationManager
29             manager.createNotificationChannel(channel)
30         }
31     }
32
33     fun showSyncSuccess(context: Context) {
34         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
35             val granted = ContextCompat.checkSelfPermission(
36                 context,
37                 Manifest.permission.POST_NOTIFICATIONS
38             ) == PackageManager.PERMISSION_GRANTED
39             if (!granted) return
40         }
41
42         val notification = NotificationCompat.Builder(context, CHANNEL_ID)
43             .setSmallIcon(R.drawable.ic_sync)
44             .setContentTitle(context.getString(R.string.
notification_sync_title))
45             .setContentText(context.getString(R.string.
notification_sync_body))
46             .setAutoCancel(true)
47             .build()
48
49         NotificationManagerCompat.from(context).notify(1001, notification)
50     }

```

Листинг 8 – Показ уведомления о синхронизации

5 ДИАГРАММА СЦЕНАРИЯ ОБНОВЛЕНИЯ ЛЕНТЫ

На рисунке 2 приведена упрощённая диаграмма последовательностей сценария «Пользователь обновляет ленту свайпом».

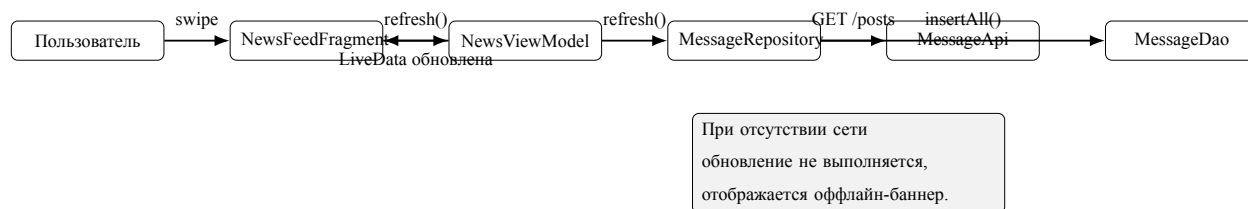


Рисунок 2 – Сценарий обновления ленты сообщений

6 СБОРКА И ЗАПУСК

Проект собирается стандартными средствами Gradle. Для удобства добавлены команды Makefile и скрипты сборки/установки.

- Сборка debug APK: `make build` (или `./build_debug.sh`).
- Установка на устройство: `make install` (нужен adb).
- Запуск приложения: `make run`.

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы реализовано учебное Android-приложение, демонстрирующее ключевые аспекты мобильной разработки: навигацию, MVVM, сетевую работу (Retrofit), локальное кэширование (Room), корутины, обработку сети, фоновые задачи (WorkManager) и уведомления. Проект может служить базой для дальнейшего расширения функционала (авторизация, реальный backend, чаты, push-уведомления и т.п.).

СПИСОК ЛИТЕРАТУРЫ

1. Android Developers. Navigation component. <https://developer.android.com/guide/navigation>
2. Android Developers. Room persistence library. <https://developer.android.com/training/data-storage/room>
3. Android Developers. WorkManager. <https://developer.android.com/topic/libraries/architecture/workmanager>
4. Square. Retrofit. <https://square.github.io/retrofit/>

ПРИЛОЖЕНИЕ А. Фрагменты исходного кода

A.1 MainActivity (навигация + разрешения)

```
1 package com.example.messenger
2
3 import android.os.Bundle
4 import android.util.Log
5 import android.Manifest
6 import android.content.pm.PackageManager
7 import android.os.Build
8 import androidx.activity.result.contract.ActivityResultContracts
9 import androidx.appcompat.app.AppCompatActivity
10 import androidx.core.content.ContextCompat
11 import androidx.navigation.ui.AppBarConfiguration
12 import androidx.navigation.ui.setupActionBarWithNavController
13 import androidx.navigation.fragment.NavHostFragment
14 import androidx.navigation.ui.setupWithNavController
15 import com.example.messenger.databinding.ActivityMainBinding
16
17 class MainActivity : AppCompatActivity() {
18
19     private lateinit var binding: ActivityMainBinding
20
21     companion object {
22         private const val TAG = "MainActivity"
23     }
24
25     override fun onCreate(savedInstanceState: Bundle?) {
26         super.onCreate(savedInstanceState)
27         Log.d(TAG, "onCreate: Activity создана")
28
29         binding = ActivityMainBinding.inflate(layoutInflater)
30         setContentView(binding.root)
31
32         setSupportActionBar(binding.toolbar)
33         setupNavigation()
34         requestAppPermissions()
35     }
36
```

```

37     private val permissionsLauncher = registerForActivityResult(
38         ActivityResultContracts.RequestMultiplePermissions()
39     ) { }
40
41     private fun requestAppPermissions() {
42         val permissions = mutableListOf<String>()
43
44         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
45             permissions.add(Manifest.permission.POST_NOTIFICATIONS)
46         }
47         permissions.add(Manifest.permission.READ_CONTACTS)
48
49         val toRequest = permissions.filter {
50             ContextCompat.checkSelfPermission(this, it) != PackageManager.
51             PERMISSION_GRANTED
52         }
53
54         if (toRequest.isNotEmpty()) {
55             permissionsLauncher.launch(toRequest.toTypedArray())
56         }
57
58     private fun setupNavigation() {
59         val navHostFragment = supportFragmentManager
60             .findFragmentById(R.id.nav_host_fragment) as NavHostFragment
61         val navController = navHostFragment.navController
62
63         val appBarConfiguration = AppBarConfiguration(
64             setOf(
65                 R.id.navigation_news,
66                 R.id.navigation_profile,
67                 R.id.navigation_settings
68             )
69         )
70         setupActionBarWithNavController(navController, appBarConfiguration)
71         binding.bottomNavigation.setupWithNavController(navController)
72         Log.d(TAG, "setupNavigation: Навигация настроена")
73     }
74

```

```

75     override fun onSupportNavigateUp(): Boolean {
76         val navHostFragment = supportFragmentManager
77             .findFragmentById(R.id.nav_host_fragment) as NavHostFragment
78         val navController = navHostFragment.navController
79         return navController.navigateUp() || super.onSupportNavigateUp()
80     }
81
82     override fun onStart() {
83         super.onStart()
84         Log.d(TAG, "onStart: Activity запущена")
85     }
86
87     override fun onResume() {
88         super.onResume()
89         Log.d(TAG, "onResume: Activity возобновлена")
90     }
91
92     override fun onPause() {
93         super.onPause()
94         Log.d(TAG, "onPause: Activity приостановлена")
95     }
96
97     override fun onStop() {
98         super.onStop()
99         Log.d(TAG, "onStop: Activity остановлена")
100    }
101
102    override fun onDestroy() {
103        super.onDestroy()
104        Log.d(TAG, "onDestroy: Activity уничтожена")
105    }
106
107    override fun onRestart() {
108        super.onRestart()
109        Log.d(TAG, "onRestart: Activity перезапущена")
110    }
111 }

```

Листинг 9 – MainActivity.kt

A.2 NewsFeedFragment (UI ленты)

```

1 package com.example.messenger.ui.news
2
3 import android.content.Context
4 import android.os.Bundle
5 import android.util.Log
6 import android.view.LayoutInflater
7 import android.view.View
8 import android.view.ViewGroup
9 import android.widget.Toast
10 import androidx.fragment.app.Fragment
11 import androidx.lifecycle.ViewModelProvider
12 import androidx.core.view.isVisible
13 import androidx.recyclerview.widget.LinearLayoutManager
14 import com.google.android.material.snackbar.Snackbar
15 import com.example.messenger.R
16 import com.example.messenger.data.api.RetrofitClient
17 import com.example.messenger.data.db.AppDatabase
18 import com.example.messenger.data.repository.MessageRepository
19 import com.example.messenger.databinding.FragmentNewsFeedBinding
20
21 class NewsFeedFragment : Fragment() {
22
23     private var _binding: FragmentNewsFeedBinding? = null
24     private val binding get() = _binding!!
25
26     private lateinit var viewModel: NewsViewModel
27     private lateinit var adapter: MessageAdapter
28
29     companion object {
30         private const val TAG = "NewsFeedFragment"
31     }
32
33     override fun onAttach(context: Context) {
34         super.onAttach(context)
35         Log.d(TAG, "onAttach: Fragment прикреплён к активности")
36     }
37
38     override fun onCreate(savedInstanceState: Bundle?) {
39         super.onCreate(savedInstanceState)
40         Log.d(TAG, "onCreate: Fragment создан")

```



```

41     }
42
43     override fun onCreateView(
44         inflater: LayoutInflater,
45         container: ViewGroup?,
46         savedInstanceState: Bundle?
47     ): View {
48         Log.d(TAG, "onCreateView: Создание View для Fragment")
49         _binding = FragmentNewsFeedBinding.inflate(inflater, container,
false)
50         return binding.root
51     }
52
53     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
54         super.onViewCreated(view, savedInstanceState)
55         Log.d(TAG, "onViewCreated: View создан")
56
57         // Initialize Data Layer
58         val database = AppDatabase.getDatabase(requireContext())
59         val repository = MessageRepository(database.messageDao(),
RetrofitClient.api)
60         val factory = NewsViewModelFactory(requireActivity().application,
repository)
61         viewModel = ViewModelProvider(this, factory)[NewsViewModel::class.
java]
62
63         // Setup RecyclerView
64         adapter = MessageAdapter { message ->
65             viewModel.toggleLike(message)
66         }
67         binding.rvMessages.layoutManager = LinearLayoutManager(context)
68         binding.rvMessages.adapter = adapter
69
70         // Observe Data
71         viewModel.messages.observe(viewLifecycleOwner) { messages ->
72             adapter.submitList(messages)
73         }
74
75         viewModel.isOnline.observe(viewLifecycleOwner) { isOnline ->
76             binding.offlineBanner.isVisible = !isOnline

```

```

77         binding.swipeRefreshLayout.isEnabled = isOnline
78         if (!isOnline) {
79             Snackbar.make(binding.root, getString(R.string.
offline_message), Snackbar.LENGTH_SHORT).show()
80         }
81     }
82
83     viewModel.isLoading.observe(viewLifecycleOwner) { isLoading ->
84         binding.swipeRefreshLayout.isRefreshing = isLoading
85     }
86
87     // Setup Listeners
88     binding.swipeRefreshLayout.setOnRefreshListener {
89         viewModel.refreshMessages()
90     }
91
92     binding.fabNewMessage.setOnClickListener {
93         Toast.makeText(requireContext(), getString(R.string.
new_message), Toast.LENGTH_SHORT).show()
94     }
95
96     // Initial load if empty (optional, but good UX)
97     if (viewModel.messages.value.isNullOrEmpty()) {
98         viewModel.refreshMessages()
99     }
100 }
101
102 override fun onStart() {
103     super.onStart()
104     Log.d(TAG, "onStart: Fragment запущен")
105 }
106
107 override fun onResume() {
108     super.onResume()
109     Log.d(TAG, "onResume: Fragment возобновлён")
110 }
111
112 override fun onPause() {
113     super.onPause()
114     Log.d(TAG, "onPause: Fragment приостановлен")

```

```

115     }
116
117     override fun onStop() {
118         super.onStop()
119         Log.d(TAG, "onStop: Fragment остановлен")
120     }
121
122     override fun onDestroyView() {
123         super.onDestroyView()
124         Log.d(TAG, "onDestroyView: View уничтожен")
125         _binding = null
126     }
127
128     override fun onDestroy() {
129         super.onDestroy()
130         Log.d(TAG, "onDestroy: Fragment уничтожен")
131     }
132
133     override fun onDetach() {
134         super.onDetach()
135         Log.d(TAG, "onDetach: Fragment откреплён от активности")
136     }
137 }

```

Листинг 10 – NewsFeedFragment.kt

A.3 NewsViewModel (обновление и лайки)

```

1 package com.example.messenger.ui.news
2
3 import android.app.Application
4 import androidx.lifecycle.AndroidViewModel
5 import androidx.lifecycle.LiveData
6 import androidx.lifecycle.MutableLiveData
7 import androidx.lifecycle.ViewModel
8 import androidx.lifecycle.ViewModelProvider
9 import androidx.lifecycle.asLiveData
10 import androidx.lifecycle.viewModelScope
11 import com.example.messenger.data.model.Message
12 import com.example.messenger.data.network.NetworkStatusTracker
13 import com.example.messenger.data.repository.MessageRepository
14 import kotlinx.coroutines.launch

```

```

15
16 class NewsViewModel(
17     application: Application,
18     private val repository: MessageRepository
19 ) : AndroidViewModel(application) {
20
21     private val networkStatusTracker = NetworkStatusTracker(application)
22     val isOnline: LiveData<Boolean> = networkStatusTracker.isOnline.
asLiveData()
23
24     val messages = repository.messages.asLiveData()
25
26     private val _isLoading = MutableLiveData<Boolean>()
27     val isLoading: LiveData<Boolean> = _isLoading
28
29     fun refreshMessages() {
30         viewModelScope.launch {
31             _isLoading.value = true
32             try {
33                 if (isOnline.value != false) {
34                     repository.refreshMessages()
35                 }
36             } catch (e: Exception) {
37                 // Handle error (e.g., show toast via LiveData event)
38             } finally {
39                 _isLoading.value = false
40             }
41         }
42     }
43
44     fun toggleLike(message: Message) {
45         viewModelScope.launch {
46             repository.toggleLike(message)
47         }
48     }
49 }
50
51 class NewsViewModelFactory(
52     private val application: Application,
53     private val repository: MessageRepository

```

```

54 ) : ViewModelProvider.Factory {
55     override fun <T : ViewModel> create(modelClass: Class<T>): T {
56         if (modelClass.isAssignableFrom(NewsViewModel::class.java)) {
57             @Suppress("UNCHECKED_CAST")
58             return NewsViewModel(application, repository) as T
59         }
60         throw IllegalArgumentException("Unknown ViewModel class")
61     }
62 }

```

Листинг 11 – NewsViewModel.kt

A.4 Репозиторий и локальный кэш (Room)

```

1 package com.example.messenger.data.repository
2
3 import com.example.messenger.data.api.MessageApi
4 import com.example.messenger.data.db.MessageDao
5 import com.example.messenger.data.model.Message
6 import kotlinx.coroutines.flow.Flow
7
8 class MessageRepository(
9     private val messageDao: MessageDao,
10    private val messageApi: MessageApi
11 ) {
12    val messages: Flow<List<Message>> = messageDao.getAllMessages()
13
14    suspend fun refreshMessages() {
15        try {
16            val localLikes = messageDao.getMessageLikes().associateBy { it
17                .id }
18            val remoteMessages = messageApi.getMessages()
19            val mergedMessages = remoteMessages.map { message ->
20                val isLiked = localLikes[message.id]?.isLiked ?: false
21                message.copy(isLiked = isLiked)
22            }
23            messageDao.insertAll(mergedMessages)
24        } catch (e: Exception) {
25            // Handle error or rethrow. For now, we just log or ignore to
26            keep UI simple,
27            // but in a real app we'd expose an error state.
28            e.printStackTrace()
29        }
30    }
31 }

```

```

27         throw e
28     }
29 }
30
31 suspend fun toggleLike(message: Message) {
32     messageDao.updateLike(message.id, !message.isLiked)
33 }
34 }

```

Листинг 12 – MessageRepository.kt

```

1 package com.example.messenger.data.db
2
3 import androidx.room.Dao
4 import androidx.room.Insert
5 import androidx.room.OnConflictStrategy
6 import androidx.room.Query
7 import com.example.messenger.data.model.Message
8 import kotlinx.coroutines.flow.Flow
9
10 @Dao
11 interface MessageDao {
12     @Query("SELECT * FROM messages")
13     fun getAllMessages(): Flow<List<Message>>
14
15     @Query("SELECT id, isLiked FROM messages")
16     suspend fun getMessageLikes(): List<MessageLike>
17
18     @Insert(onConflict = OnConflictStrategy.REPLACE)
19     suspend fun insertAll(messages: List<Message>)
20
21     @Query("UPDATE messages SET isLiked = :isLiked WHERE id = :id")
22     suspend fun updateLike(id: Int, isLiked: Boolean)
23 }

```

Листинг 13 – MessageDao.kt

```

1 package com.example.messenger.data.db
2
3 import android.content.Context
4 import androidx.room.Database
5 import androidx.room.Room
6 import androidx.room.RoomDatabase

```

```

7 import com.example.messenger.data.model.Message
8
9 @Database(entities = [Message::class], version = 2, exportSchema = false)
10 abstract class AppDatabase : RoomDatabase() {
11     abstract fun messageDao(): MessageDao
12
13     companion object {
14         @Volatile
15         private var INSTANCE: AppDatabase? = null
16
17         fun getDatabase(context: Context): AppDatabase {
18             return INSTANCE ?: synchronized(this) {
19                 val instance = Room.databaseBuilder(
20                     context.applicationContext,
21                     AppDatabase::class.java,
22                     "messenger_database"
23                 ).fallbackToDestructiveMigration().build()
24                 INSTANCE = instance
25                 instance
26             }
27         }
28     }
29 }

```

Листинг 14 – AppDatabase.kt

A.5 Retrofit API

```

1 package com.example.messenger.data.api
2
3 import retrofit2.Retrofit
4 import retrofit2.converter.gson.GsonConverterFactory
5
6 object RetrofitClient {
7     private const val BASE_URL = "https://jsonplaceholder.typicode.com/"
8
9     val api: MessageApi by lazy {
10         Retrofit.Builder()
11             .baseUrl(BASE_URL)
12             .addConverterFactory(GsonConverterFactory.create())
13             .build()
14             .create(MessageApi::class.java)

```

```
15     }  
16 }
```

Листинг 15 – RetrofitClient.kt

```
1 package com.example.messenger.data.api  
2  
3 import com.example.messenger.data.model.Message  
4 import retrofit2.http.GET  
5  
6 interface MessageApi {  
7     @GET("posts")  
8     suspend fun getMessages(): List<Message>  
9 }
```

Листинг 16 – MessageApi.kt

A.6 Сеть/фон/уведомления

```
1 package com.example.messenger.data.network  
2  
3 import android.content.Context  
4 import android.net.ConnectivityManager  
5 import android.net.Network  
6 import android.net.NetworkCapabilities  
7 import android.net.NetworkRequest  
8 import kotlinx.coroutines.channels.awaitClose  
9 import kotlinx.coroutines.flow.Flow  
10 import kotlinx.coroutines.flow.callbackFlow  
11 import kotlinx.coroutines.flow.distinctUntilChanged  
12  
13 class NetworkStatusTracker(context: Context) {  
14     private val connectivityManager =  
15         context.getSystemService(Context.CONNECTIVITY_SERVICE) as  
16         ConnectivityManager  
17  
18     val isOnline: Flow<Boolean> = callbackFlow {  
19         val callback = object : ConnectivityManager.NetworkCallback() {  
20             override fun onAvailable(network: Network) {  
21                 trySend(true)  
22             }  
23  
24             override fun onLost(network: Network) {
```



```

24         trySend(false)
25     }
26 }
27
28     val activeNetwork = connectivityManager.activeNetwork
29     val capabilities = connectivityManager.getNetworkCapabilities(
    activeNetwork)
30     val connected = capabilities?.hasCapability(NetworkCapabilities.
    NET_CAPABILITY_INTERNET) == true
31     trySend(connected)
32
33     val request = NetworkRequest.Builder()
34         .addCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET)
35         .build()
36
37     connectivityManager.registerNetworkCallback(request, callback)
38
39     awaitClose {
40         connectivityManager.unregisterNetworkCallback(callback)
41     }
42 }.distinctUntilChanged()
43 }

```

Листинг 17 – NetworkStatusTracker.kt

```

1 package com.example.messenger.data.work
2
3 import android.content.Context
4 import androidx.work.Constraints
5 import androidx.work.ExistingPeriodicWorkPolicy
6 import androidx.work.NetworkType
7 import androidx.work.PeriodicWorkRequestBuilder
8 import androidx.work.WorkManager
9 import java.util.concurrent.TimeUnit
10
11 object WorkScheduler {
12     private const val WORK_NAME = "message_sync_work"
13
14     fun scheduleMessageSync(context: Context) {
15         val constraints = Constraints.Builder()
16             .setRequiredNetworkType(NetworkType.CONNECTED)
17             .build()

```

```

18
19     val request = PeriodicWorkRequestBuilder<MessageSyncWorker>(15,
    TimeUnit.MINUTES)
20         .setConstraints(constraints)
21         .build()
22
23     WorkManager.getInstance(context)
24         .enqueueUniquePeriodicWork(WORK_NAME,
    ExistingPeriodicWorkPolicy.UPDATE, request)
25 }
26 }

```

Листинг 18 – WorkScheduler.kt

```

1 package com.example.messenger.data.work
2
3 import android.content.Context
4 import androidx.work.CoroutineWorker
5 import androidx.work.WorkerParameters
6 import com.example.messenger.data.api.RetrofitClient
7 import com.example.messenger.data.db.AppDatabase
8 import com.example.messenger.data.notification.NotificationHelper
9 import com.example.messenger.data.repository.MessageRepository
10
11 class MessageSyncWorker(
12     appContext: Context,
13     params: WorkerParameters
14 ) : CoroutineWorker(appContext, params) {
15
16     override suspend fun doWork(): Result {
17         val database = AppDatabase.getDatabase(applicationContext)
18         val repository = MessageRepository(database.messageDao(),
    RetrofitClient.api)
19
20         return try {
21             repository.refreshMessages()
22             NotificationHelper.showSyncSuccess(applicationContext)
23             Result.success()
24         } catch (e: Exception) {
25             Result.retry()
26         }
27     }

```

Листинг 19 – MessageSyncWorker.kt

```

1 package com.example.messenger.data.notification
2
3 import android.Manifest
4 import android.app.NotificationChannel
5 import android.app.NotificationManager
6 import android.content.Context
7 import android.content.pm.PackageManager
8 import android.os.Build
9 import androidx.core.app.NotificationCompat
10 import androidx.core.app.NotificationManagerCompat
11 import androidx.core.content.ContextCompat
12 import com.example.messenger.R
13
14 object NotificationHelper {
15     const val CHANNEL_ID = "sync_channel"
16     private const val CHANNEL_NAME = "Синхронизация"
17     private const val CHANNEL_DESCRIPTION = "Уведомления о синхронизации
18     сообщений"
19
20     fun createChannel(context: Context) {
21         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
22             val channel = NotificationChannel(
23                 CHANNEL_ID,
24                 CHANNEL_NAME,
25                 NotificationManager.IMPORTANCE_DEFAULT
26             ).apply {
27                 description = CHANNEL_DESCRIPTION
28             }
29             val manager = context.getSystemService(Context.
30                 NOTIFICATION_SERVICE) as NotificationManager
31             manager.createNotificationChannel(channel)
32         }
33     }
34
35     fun showSyncSuccess(context: Context) {
36         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
37             val granted = ContextCompat.checkSelfPermission(
38                 context,

```

```

37         Manifest.permission.POST_NOTIFICATIONS
38     ) == PackageManager.PERMISSION_GRANTED
39     if (!granted) return
40 }
41
42     val notification = NotificationCompat.Builder(context, CHANNEL_ID)
43         .setSmallIcon(R.drawable.ic_sync)
44         .setContentTitle(context.getString(R.string.
notification_sync_title))
45         .setContentText(context.getString(R.string.
notification_sync_body))
46         .setAutoCancel(true)
47         .build()
48
49     NotificationManagerCompat.from(context).notify(1001, notification)
50 }
51 }

```

Листинг 20 – NotificationHelper.kt

A.7 Навигационный граф

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <navigation xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:app="http://schemas.android.com/apk/res-auto"
4     xmlns:tools="http://schemas.android.com/tools"
5     android:id="@+id/nav_graph"
6     app:startDestination="@id/navigation_news">
7
8     <fragment
9         android:id="@+id/navigation_news"
10        android:name="com.example.messenger.ui.news.NewsFeedFragment"
11        android:label="@string/news_feed_title"
12        tools:layout="@layout/fragment_news_feed" />
13
14    <fragment
15        android:id="@+id/navigation_profile"
16        android:name="com.example.messenger.ui.profile.ProfileFragment"
17        android:label="@string/profile_title"
18        tools:layout="@layout/fragment_profile" />
19
20    <fragment

```

```
21         android:id="@+id/navigation_settings"
22         android:name="com.example.messenger.ui.settings.SettingsFragment"
23         android:label="@string/settings_title"
24         tools:layout="@layout/fragment_settings" />
25 </navigation>
```

Листинг 21 – nav_graph.xml